

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO

Bases de Datos I

Red Social Pascualina

Tarea 2 — Modelo Lógico, Normalización e Implementación en PostgreSQL

Integrantes:

Emanuel Ortiz

Saray Muñoz

Medellín, septiembre de 2026

1. Justificación General del Diseño

El objetivo principal de esta base de datos es proporcionar la infraestructura técnica para la Red Social Estudiantil Pascualina, soportando una comunicación fluida, intercambio de información académica, interacción social e integración mediante grupos y eventos.

Para lograr esto de forma eficiente y escalable en PostgreSQL, se transformó el modelo conceptual inicial en un modelo relacional lógico, aplicando reglas estrictas de integridad referencial, normalización hasta la Tercera Forma Normal (3FN) y optimización del almacenamiento de datos.

El proyecto se organizó mediante un repositorio en GitHub con una estructuración estricta de carpetas (/informe, /video), documentación en el archivo README.md y la grabación de un video explicativo, fortaleciendo así las habilidades de comunicación técnica y gestión de proyectos del equipo.

2. Proceso y Justificación de la Normalización (1FN, 2FN, 3FN)

La normalización es el proceso de organizar los datos de una base de datos para reducir la redundancia y mejorar la integridad. A continuación se explica cómo se aplicó cada una de las tres formas normales al modelo de la Red Social Pascualina.

2.1 Primera Forma Normal (1FN) — Atomicidad de los datos

Una tabla está en 1FN cuando todos sus atributos contienen valores atómicos (indivisibles), no hay grupos repetidos y existe una clave primaria que identifica cada fila de manera única.

Problema antes de aplicar 1FN — tabla sin normalizar:

id_usuario	nombre	email	intereses
1	Camilo Pérez	camilo@pascualbravo.edu.co	Programación, Bases de Datos, Python
2	Maria Gómez	maria@pascualbravo.edu.co	UI/UX, Ilustración, Diseño web

El campo 'intereses' contiene múltiples valores en una sola celda — eso viola la 1FN porque no es atómico.

Solución aplicada — manejo de campos multivaluados:

- Atributos como los intereses o habilidades de los estudiantes no se almacenan como listas anidadas o arrays complejos dentro de la tabla principal que comprometan la consistencia, sino como campos TEXT de consulta general.
- Claves primarias: cada tabla cuenta con una clave primaria única (PRIMARY KEY) definida mediante campos tipo SERIAL (enteros autoincrementables), asegurando que cada registro sea identificable de forma inequívoca.

Las 6 tablas del sistema cumplen la 1FN:

Tabla	Clave Primaria	¿1FN?	Justificación
usuarios	id_usuario SERIAL	✓ Sí	Cada fila es única, todos los campos son atómicos
publicaciones	id_publicacion SERIAL	✓ Sí	Cada publicación tiene un ID único e irrepetible
grupos	id_grupo SERIAL	✓ Sí	Cada grupo tiene un ID único
eventos	id_evento SERIAL	✓ Sí	Cada evento tiene un ID único
seguidores	id_seguidor + id_seguido	✓ Sí	Clave primaria compuesta garantiza unicidad
miembros_grupo	id_grupo + id_usuario	✓ Sí	Clave primaria compuesta garantiza unicidad

2.2 Segunda Forma Normal (2FN) — Eliminación de dependencias parciales

Una tabla está en 2FN cuando está en 1FN y todos los atributos no clave dependen completamente de la clave primaria, no de una parte de ella. Esto aplica principalmente a tablas con claves primarias compuestas.

- En la tabla publicaciones, el contenido y la fecha de creación dependen directamente del id único de la publicación (id_publicacion).
- En las tablas intermedias de relaciones muchos a muchos (seguidores y miembros_grupo), la clave es compuesta. Atributos propios de la relación como fecha_conexion o rol dependen de la combinación completa de ambas claves foráneas, no de una sola parte.

Tabla	Clave Compuesta	Atributo	¿Depende de toda la clave? / ¿2FN?
seguidores	id_seguidor + id_seguido	fecha_conexion	Sí, depende de la combinación de ambos — ✓ Cumple 2FN
miembros_grupo	id_grupo + id_usuario	rol	Sí, depende de quién está en qué grupo — ✓ Cumple 2FN
miembros_grupo	id_grupo + id_usuario	fecha_ingreso	Sí, depende de la combinación completa — ✓ Cumple 2FN

Todas las tablas cumplen la 2FN porque los atributos propios de cada relación dependen de la combinación completa de las claves foráneas, no de una sola parte.

2.3 Tercera Forma Normal (3FN) — Eliminación de dependencias transitivas

Una tabla está en 3FN cuando está en 2FN y no existen dependencias transitivas — es decir, ningún atributo no clave depende de otro atributo no clave.

Problema antes de aplicar 3FN — ejemplo de dependencia transitiva:

id_publicacion	id_usuario_autor	nombre_autor	email_autor	contenido
1	1	Camilo Pérez	camilo@pascualbravo.edu.co	¿A qué hora es la clase?

En este caso, nombre_autor y email_autor dependen de id_usuario_autor, no de id_publicacion directamente. Eso es una dependencia transitiva que viola la 3FN.

Solución aplicada — tabla publicaciones en 3FN:

id_publicacion	id_usuario_autor (FK)	contenido	categoria	fecha_creacion
1	1	¿A qué hora es la clase de Bases de Datos?	Consulta	2026-09-26 11:36

Se eliminó nombre_autor y email_autor de la tabla publicaciones. Si se necesitan esos datos, se obtienen mediante un JOIN con la tabla usuarios. Además, únicamente se almacena el campo id_usuario_autor (clave foránea): no se guarda el nombre ni el correo del usuario en la tabla de publicaciones, evitando redundancia. Así, si un usuario cambia su nombre o correo, solo se actualiza en un lugar y la consistencia se mantiene automáticamente en todas las tablas relacionadas.

3. Justificación de los Tipos de Datos y Longitudes

La elección de cada tipo de dato buscó equilibrar el rendimiento, el uso de espacio en disco y la flexibilidad para almacenar información de longitud variable:

Campo / Tabla	Tipo de Dato Implementado	Justificación Técnica
id_* (Claves Primarias)	SERIAL / INT	Proporciona un identificador numérico entero de 4 bytes, altamente eficiente para operaciones de indexación y uniones (JOINS) en PostgreSQL.

Campo / Tabla	Tipo de Dato Implementado	Justificación Técnica
nombre / titulo	VARCHAR(100) / VARCHAR(150)	Permite almacenar cadenas de caracteres de longitud variable, optimizando el uso de memoria en disco frente a tipos fijos como CHAR.
email	VARCHAR(150) UNIQUE	Garantiza espacio suficiente para direcciones institucionales largas y restringe duplicados a nivel de motor mediante la cláusula UNIQUE.
contenido / descripcion	TEXT	Asigna almacenamiento dinámico sin límite estricto prefijado, ideal para publicaciones, comentarios o descripciones extensas de grupos y eventos.
fecha_*	TIMESTAMP	Registra la fecha y hora exactas en que se crea una publicación, registro o evento, esencial para auditoría y ordenamiento cronológico.

4. Modelo Lógico — Tablas y Estructura

A continuación se presenta la estructura lógica completa de la base de datos, con las 6 tablas que conforman el sistema.

4.1 Tabla: usuarios

Almacena la información del perfil de cada estudiante registrado en la plataforma.

Atributo	Tipo de dato	Restricción	Descripción	Justificación
id_usuario	SERIAL / Entero	PK	Identificador único del usuario	Entero autoincremental eficiente para JOINS e indexación
nombre	VARCHAR(100)	NOT NULL	Nombre completo del estudiante	VARCHAR optimiza memoria vs CHAR en nombres de longitud variable
email	VARCHAR(150)	UNIQUE NOT NULL	Correo institucional único	UNIQUE evita duplicados a nivel de motor de base de datos
area_estudio	VARCHAR(100)	Opcional	Programa o área académica	Permite filtrar estudiantes por carrera o interés académico
intereses	TEXT	Opcional	Descripción de intereses del estudiante	TEXT sin límite para descripciones extensas y flexibles
habilidades	TEXT	Opcional	Habilidades técnicas o deportivas	TEXT permite describir múltiples habilidades en formato libre
fecha_registro	TIMESTAMP	DEFAULT NOW()	Fecha y hora de registro	Registra el momento exacto para auditoría cronológica

4.2 Tabla: publicaciones

Almacena el contenido que los estudiantes comparten en la plataforma.

Atributo	Tipo de dato	Restricción	Descripción	Justificación
id_publicacion	SERIAL / Entero	PK	Identificador único de la publicación	Autoincremental garantiza unicidad sin intervención manual
id_usuario_autor	INT / Entero	FK → usuarios	Referencia al autor de la publicación	FK garantiza que solo usuarios existentes puedan publicar
contenido	TEXT	NOT NULL	Texto de la publicación	TEXT sin límite para publicaciones de cualquier extensión
categoria	VARCHAR(50)	Opcional	Categoría de la publicación	Permite clasificar publicaciones por tipo
fecha_creacion	TIMESTAMP	DEFAULT NOW()	Fecha y hora de la publicación	Esencial para ordenar publicaciones cronológicamente

4.3 Tabla: seguidores

Tabla intermedia que gestiona la relación N:M entre usuarios — quién sigue a quién.

Atributo	Tipo de dato	Restricción	Descripción	Justificación
id_seguidor	INT / Entero	PK + FK → usuarios	Usuario que sigue a otro	Parte de la clave compuesta que identifica la relación

Atributo	Tipo de dato	Restricción	Descripción	Justificación
id_seguido	INT / Entero	PK + FK → usuarios	Usuario que es seguido	Parte de la clave compuesta que identifica la relación
fecha_conexion	TIMESTAMP	DEFAULT NOW()	Fecha en que se estableció la conexión	Permite saber cuándo se inició la relación de seguimiento

Restricción adicional: CHECK (id_seguidor <> id_seguido) — evita que un usuario se siga a sí mismo.

4.4 Tabla: grupos

Almacena los grupos de estudio o proyectos creados por los estudiantes.

Atributo	Tipo de dato	Restricción	Descripción	Justificación
id_grupo	SERIAL / Entero	PK	Identificador único del grupo	Autoincremental para identificación única
nombre_grupo	VARCHAR(100)	NOT NULL	Nombre del grupo de estudio	VARCHAR limita el nombre a 100 caracteres — suficiente y eficiente
descripcion	TEXT	Opcional	Descripción del propósito del grupo	TEXT permite descripciones extensas sin restricción
materia_especifica	VARCHAR(100)	Opcional	Materia relacionada con el grupo	Permite filtrar grupos por asignatura académica
fecha_creacion	TIMESTAMP	DEFAULT NOW()	Fecha de creación del grupo	Registro cronológico para auditoría

4.5 Tabla: miembros_grupo

Tabla intermedia que gestiona la relación N:M entre usuarios y grupos.

Atributo	Tipo de dato	Restricción	Descripción	Justificación
id_grupo	INT / Entero	PK + FK → grupos	Referencia al grupo	Parte de la clave compuesta
id_usuario	INT / Entero	PK + FK → usuarios	Referencia al usuario miembro	Parte de la clave compuesta
rol	VARCHAR(30)	DEFAULT 'Miembro'	Rol del usuario en el grupo	Permite diferenciar administradores de miembros regulares
fecha_ingreso	TIMESTAMP	DEFAULT NOW()	Fecha en que el usuario se unió	Registro cronológico de membresía

4.6 Tabla: eventos

Almacena los eventos académicos o sociales organizados por los estudiantes.

Atributo	Tipo de dato	Restricción	Descripción	Justificación
id_evento	SERIAL / Entero	PK	Identificador único del evento	Autoincremental para identificación única
id_creador	INT / Entero	FK → usuarios	Usuario que creó el evento	Garantiza que el creador sea un usuario registrado
titulo	VARCHAR(150)	NOT NULL	Título del evento	150 caracteres suficientes para títulos descriptivos
descripcion	TEXT	Opcional	Detalles del evento	TEXT permite descripciones extensas
fecha_evento	TIMESTAMP	NOT NULL	Fecha y hora del evento	NOT NULL porque el evento debe tener una fecha definida
lugar_o_link	VARCHAR(255)	Opcional	Lugar físico o enlace virtual	255 caracteres suficientes para URLs largas

5. Relaciones entre Tablas

Relación	Tablas involucradas	Cardinalidad	Implementación lógica	Justificación
Usuario publica	usuarios → publicaciones	1:N	id_usuario_autor como FK en publicaciones	Un usuario puede tener N publicaciones, pero cada publicación tiene un solo autor

Relación	Tablas involucradas	Cardinalidad	Implementación lógica	Justificación
Usuario sigue a Usuario	usuarios ↔ usuarios	N:M	Tabla intermedia seguidores (id_seguidor, id_seguido)	Un usuario puede seguir a N compañeros y ser seguido por N compañeros
Usuario pertenece a Grupo	usuarios ↔ grupos	N:M	Tabla intermedia miembros_grupo (id_grupo, id_usuario)	Un usuario puede estar en N grupos y cada grupo puede tener N miembros
Usuario crea Evento	usuarios → eventos	1:N	id_creador como FK en eventos	Un usuario puede crear N eventos, pero cada evento tiene un solo creador

Detalle de implementación de cada relación

A. Relación 1:N (Usuario a Publicaciones / Usuario a Eventos)

- Implementación: se incluyó el atributo id_usuario_autor (o id_creador) como clave foránea (FOREIGN KEY) dentro de la tabla dependiente.
- Justificación: un estudiante puede publicar múltiples posts o crear múltiples eventos, pero cada post o evento pertenece únicamente a un autor. Colocar la clave foránea del lado del 'N' garantiza la integridad estructural sin generar campos nulos.

B. Relación N:M (Usuario a Usuario — Seguidores)

- Implementación: creación de la tabla intermedia seguidores con clave primaria compuesta (id_seguidor, id_seguido).
- Justificación: representa la red de contactos donde un estudiante puede seguir a N compañeros y a su vez ser seguido por N compañeros. Incluye la restricción CHECK (id_seguidor <> id_seguido) para evitar que un usuario se siga a sí mismo.

C. Relación N:M (Usuario a Grupos — Miembros)

- Implementación: creación de la tabla intermedia miembros_grupo con claves foráneas cruzadas hacia usuarios y grupos.
- Justificación: permite que un estudiante pertenezca a múltiples grupos de estudio o proyectos, y que cada grupo contenga múltiples integrantes, soportando atributos propios del vínculo como el rol (Administrador, Miembro).

D. Integridad Referencial (ON DELETE CASCADE)

- Justificación: todas las claves foráneas fueron configuradas con la cláusula ON DELETE CASCADE. Esto asegura que si un usuario elimina su cuenta, se remuevan automáticamente sus publicaciones, membresías a grupos y registros de seguimiento asociados, manteniendo la base de datos libre de registros huérfanos.

6. Diagrama del Modelo Lógico

El diagrama muestra las 6 tablas con sus atributos, claves primarias (PK), claves foráneas (FK) y las relaciones entre ellas, siguiendo la notación de modelo lógico relacional:

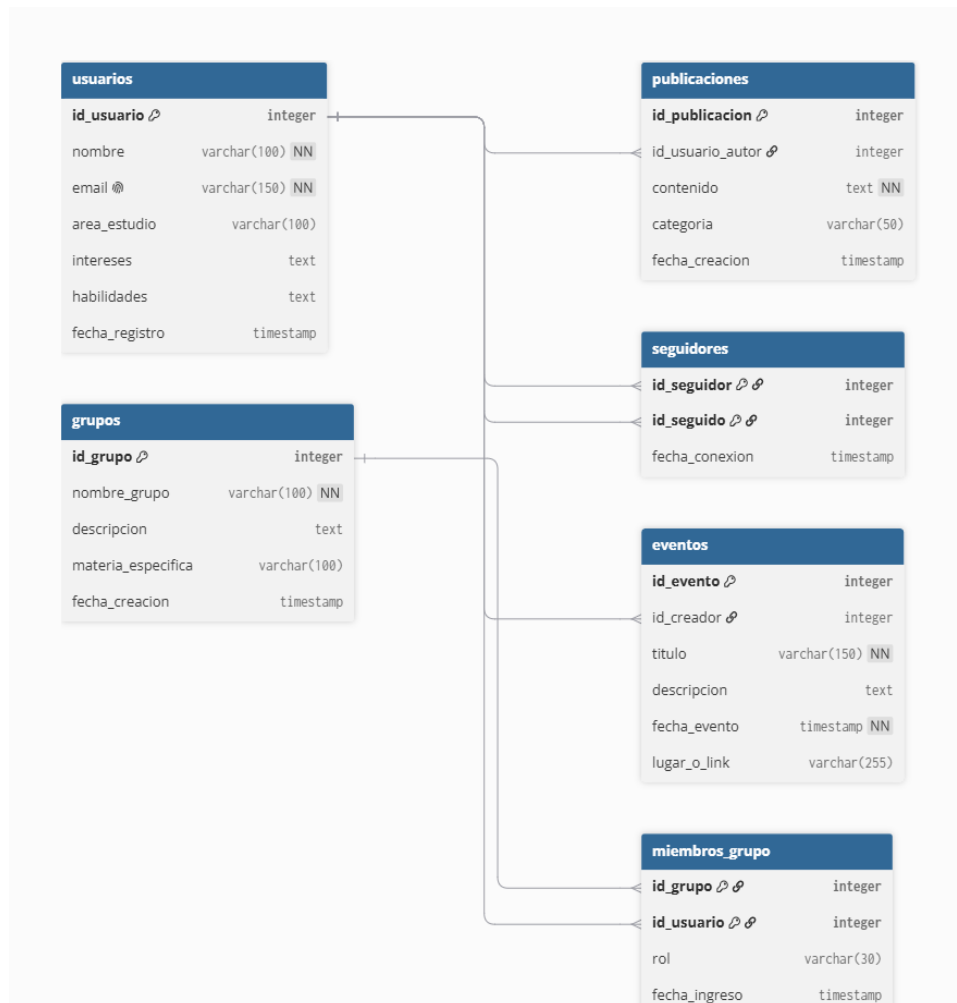


Figura 1. Diagrama del modelo lógico relacional — Red Social Pascualina

7. Implementación en PostgreSQL — Script SQL

A continuación se presenta el script SQL completo utilizado para crear las 6 tablas del sistema en pgAdmin, respetando el orden de dependencias (las tablas referenciadas se crean antes que las tablas que las referencian mediante claves foráneas).

7.1 Tabla de Usuarios (Perfiles)

```

CREATE TABLE usuarios (
    id_usuario SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    area_estudio VARCHAR(100),
    intereses TEXT,
    habilidades TEXT,
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
  
```

7.2 Tabla de Conexiones / Seguidores

```

CREATE TABLE seguidores (
    id_seguidor INT NOT NULL,
  
```

```

id_seguido INT NOT NULL,
fecha_conexion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (id_seguidor, id_seguido),
FOREIGN KEY (id_seguidor) REFERENCES usuarios(id_usuario) ON DELETE CASCADE,
FOREIGN KEY (id_seguido) REFERENCES usuarios(id_usuario) ON DELETE CASCADE,
CHECK (id_seguidor <> id_seguido)
);

```

7.3 Tabla de Publicaciones

```

CREATE TABLE publicaciones (
  id_publicacion SERIAL PRIMARY KEY,
  id_usuario_autor INT NOT NULL,
  contenido TEXT NOT NULL,
  categoria VARCHAR(50),
  fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (id_usuario_autor) REFERENCES usuarios(id_usuario) ON DELETE CASCADE
);

```

7.4 Tablas de Grupos y Miembros de Grupos

```

-- Tabla para los grupos de estudio o proyectos
CREATE TABLE grupos (
  id_grupo SERIAL PRIMARY KEY,
  nombre_grupo VARCHAR(100) NOT NULL,
  descripcion TEXT,
  materia_especifica VARCHAR(100),
  fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Tabla intermedia para saber qué usuario está en qué grupo
CREATE TABLE miembros_grupo (
  id_grupo INT NOT NULL,
  id_usuario INT NOT NULL,
  rol VARCHAR(30) DEFAULT 'Miembro',
  fecha_ingreso TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id_grupo, id_usuario),
  FOREIGN KEY (id_grupo) REFERENCES grupos(id_grupo) ON DELETE CASCADE,
  FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario) ON DELETE CASCADE
);

```

7.5 Tabla de Eventos

```

CREATE TABLE eventos (
  id_evento SERIAL PRIMARY KEY,
  id_creador INT NOT NULL,
  titulo VARCHAR(150) NOT NULL,
  descripcion TEXT,
  fecha_evento TIMESTAMP NOT NULL,
  lugar_o_link VARCHAR(255),
  FOREIGN KEY (id_creador) REFERENCES usuarios(id_usuario) ON DELETE CASCADE
);

```

8. Verificación y Comprobación de Funcionamiento

8.1 Comprobar que las tablas fueron creadas

- En el menú de la izquierda de pgAdmin, dentro de la base de datos red_social_pascualina, se despliega la ruta: Schemas → public → Tables.
- Allí se verifican las 6 tablas creadas: eventos, grupos, miembros_grupo, publicaciones, seguidores y usuarios.

8.2 Pruebas de inserción y consulta

Para verificar que el modelo funciona correctamente, se insertaron registros de prueba y se consultaron los datos:

```
-- 1. Insertar dos usuarios de prueba
INSERT INTO usuarios (nombre, email, area_estudio, intereses)
VALUES ('Camilo Pérez', 'camilo@pascualbravo.edu.co', 'Sistemas', 'Programación, Bases de Datos'),
      ('Maria Gómez', 'maria@pascualbravo.edu.co', 'Diseño', 'UI/UX, Ilustración');

-- 2. Insertar una publicación
INSERT INTO publicaciones (id_usuario_autor, contenido, categoria)
VALUES (1, '¿Alguien sabe a qué hora es la clase de Bases de Datos?', 'Consulta');

-- 3. Ver los datos
SELECT * FROM usuarios;
SELECT * FROM publicaciones;
```

Evidencia de ejecución en pgAdmin 4:

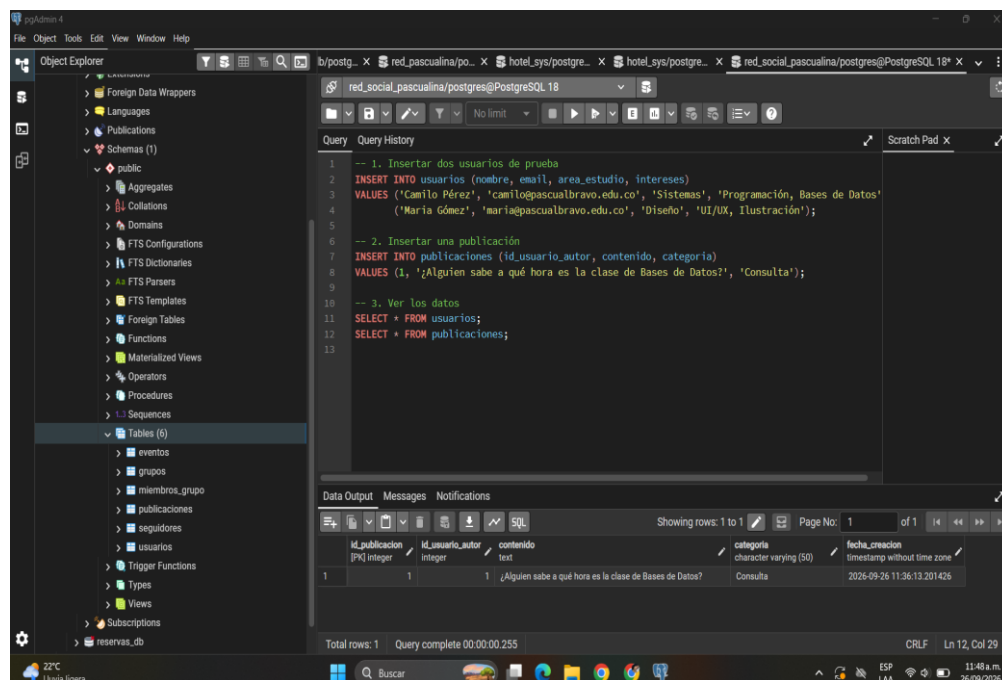


Figura 2. Ejecución del script de prueba y resultado de la consulta en pgAdmin 4

9. Conclusiones

Conclusión grupal

Como equipo, logramos llevar con éxito los requerimientos informales de la red social estudiantil a un modelo relacional eficiente en PostgreSQL. Comprobamos que el proceso de normalización hasta la Tercera Forma Normal (3FN) no es solo un ejercicio teórico, sino una herramienta práctica esencial para

evitar datos duplicados y garantizar la consistencia en el almacenamiento de publicaciones, grupos y conexiones.

La organización del proyecto mediante repositorios en GitHub nos brindó una experiencia cercana al entorno laboral real. La estructuración estricta de carpetas (/informe, /video), la documentación en el archivo README.md y la grabación del video explicativo fortalecieron nuestras habilidades de comunicación técnica y gestión de proyectos.

Saray Muñoz

Mi mayor aprendizaje fue traducir el diagrama entidad-relación a código SQL ejecutable en pgAdmin. Aprender a elegir el tipo de dato adecuado para cada atributo — como SERIAL para IDs autoincrementables y TIMESTAMP para fechas — me dio una visión clara sobre la optimización del rendimiento y el uso del espacio en el motor de base de datos.

Emanuel Ortiz

Durante esta tarea comprendí en profundidad el proceso de normalización y su importancia real en el diseño de bases de datos. La parte que más me aportó fue entender la diferencia entre la 2FN y la 3FN — al principio me parecían conceptos similares, pero al aplicarlos en las tablas de la Red Social Pascualina quedó claro que la 2FN elimina dependencias parciales en claves compuestas, mientras que la 3FN elimina dependencias entre atributos que no son clave. También aprendí que el diccionario de datos no es solo una formalidad, sino una herramienta de comunicación técnica que permite a cualquier desarrollador entender la estructura de la base de datos sin necesidad de leer el código SQL. Esta tarea me dio las bases para diseñar bases de datos más eficientes en proyectos futuros.